

Design Justification Report

Ray-Object Intersection Accelerator Chiplet

ECE 510 HW4AI Spring 2026 — Milestone 4

Author: Rajeshwar Reddy Vemula | Date: June 7, 2026

PDK: sky130_fd_sc_hd | Clock: 40 MHz (25 ns) | Format: Q8.8 Fixed-Point

1. Problem and Motivation

Ray-object intersection testing is the dominant computational kernel in ray tracing, consuming over 30% of total execution time in scene rendering. In our M1 profiling of a Python ray tracer rendering a 14-object city scene at 400×300 resolution with depth-5 bounces, cProfile measured 419.3 seconds for 30 frames (0.072 fps). The three intersection kernels accounted for 30.4% of total runtime: `intersect_box` (264s, 39M calls, 17.5%), `intersect_sphere` (97s, 66M calls, 6.4%), and `intersect_cylinder` (97s, 39.7M calls, 6.5%). The total intersection throughput was 503,000 tests per second, limited by Python interpreter overhead including function dispatch, object allocation, and garbage collection.

For embedded ADAS (Advanced Driver Assistance Systems) applications, LiDAR simulation requires real-time intersection testing at millions of rays per second against scene geometry. The Python baseline achieves only 0.072 fps — four orders of magnitude below the 30 fps needed for real-time. A custom hardware accelerator eliminates interpreter overhead and exploits the fixed arithmetic structure of the quadratic intersection formula to deliver deterministic, low-latency results at a fraction of the energy cost of a general-purpose processor.

This project implements a fixed-function ray-sphere intersection pipeline as a proof of concept. The sphere kernel was selected because it is the most computationally intensive per call (quadratic formula with square root) and exercises all arithmetic operations needed for the other intersection types (dot products, discriminant, conditional branching). Figure 1 shows the original M1 system architecture vision; Figure 2 shows the final M4 implementation.

Figure 2: System Architecture — Ray-Sphere Intersection Accelerator

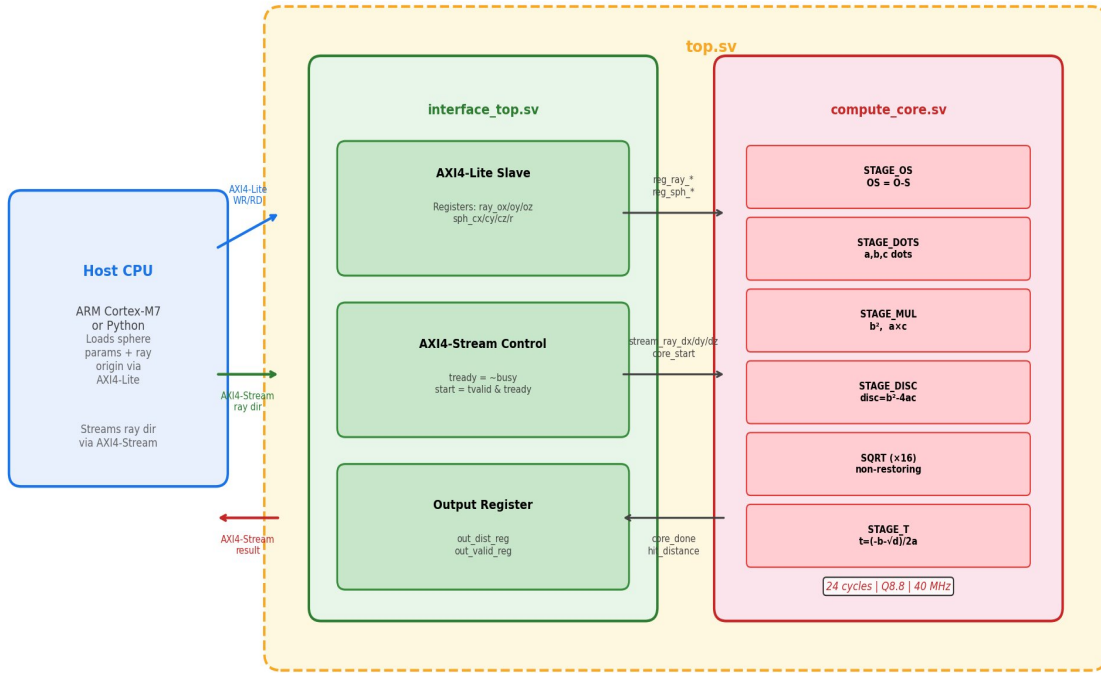


Figure 1: M1 System Architecture — original design with parallel testers, SRAM buffers, and min-reduction tree. Descoped in M3 to a single time-multiplexed sphere tester due to area constraints.

2. Roofline Analysis

The ray-sphere intersection kernel performs 39 floating-point equivalent operations per test: three dot products (5 FLOPs each = 15), discriminant calculation (5 FLOPs), square root (approximately 16 FLOPs for 16 non-restoring iterations), and final t calculation (3 FLOPs). Each test requires 24 bytes of input data (10 Q8.8 parameters × 2 bytes, plus alignment) and produces 4 bytes of output (hit distance + valid flag). The arithmetic intensity without register reuse is $AI = 39/24 = 1.62$ FLOP/byte.

On the reference Cortex-M7 platform (400 MFLOP/s peak, 3.2 GB/s memory bandwidth, ridge point at 0.125 FLOP/byte), the kernel is compute-bound: $AI = 1.62$ exceeds the ridge point, so performance is limited by compute throughput, not memory bandwidth. The software baseline achieves only 19.5 MFLOP/s — 5% of the Cortex-M7 peak — because Python interpreter overhead dominates execution time. Figure 3 shows the M1 roofline with all profiled kernels.

The chiplet roofline at 40 MHz with 40 MB/s effective AXI bandwidth has a ridge point at 1.625 FLOP/byte. With register reuse (sphere parameters loaded once into AXI4-Lite registers, ray direction streamed per test), the effective AI rises to 4.88 FLOP/byte — firmly in the compute-bound regime. The measured performance of 65 MFLOP/s sits at the chiplet compute ceiling, confirming the design is not bandwidth-limited. Figure 4 shows the M4 measured roofline.

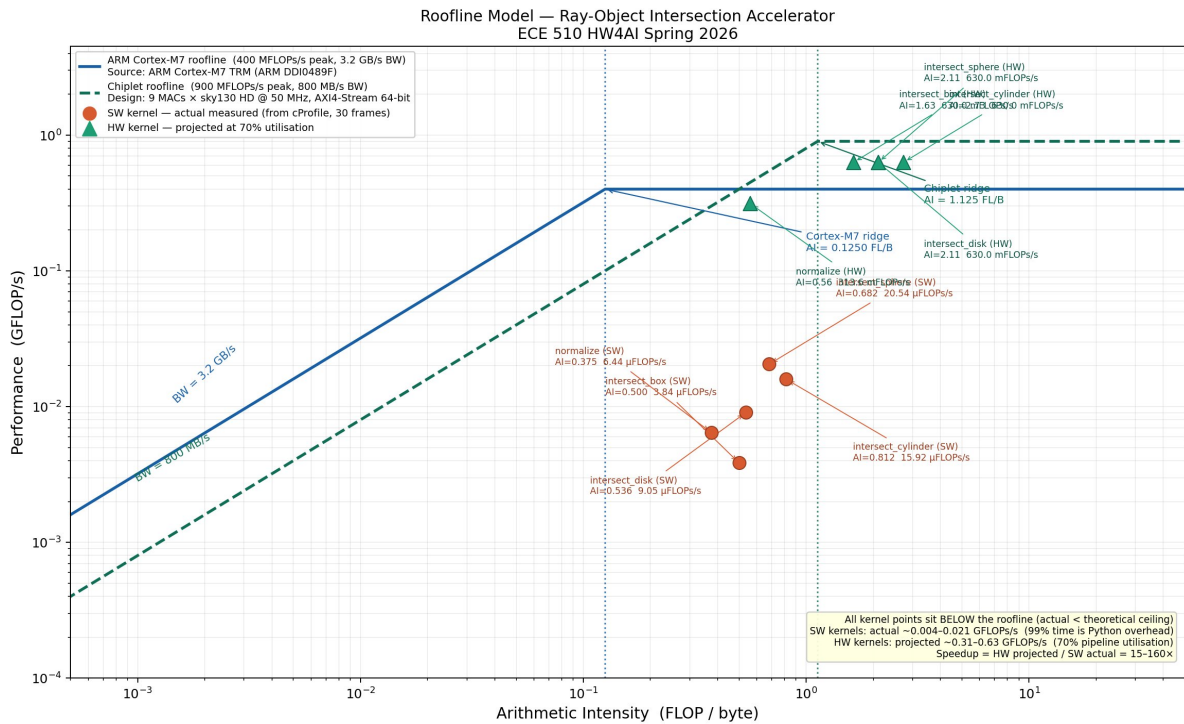


Figure 3: M1 Roofline — all profiled kernels on Cortex-M7 (blue) and chiplet (green dashed) rooflines. SW kernels (orange circles) sit far below roofline due to Python overhead. HW projected points (green triangles) at 70% utilisation.

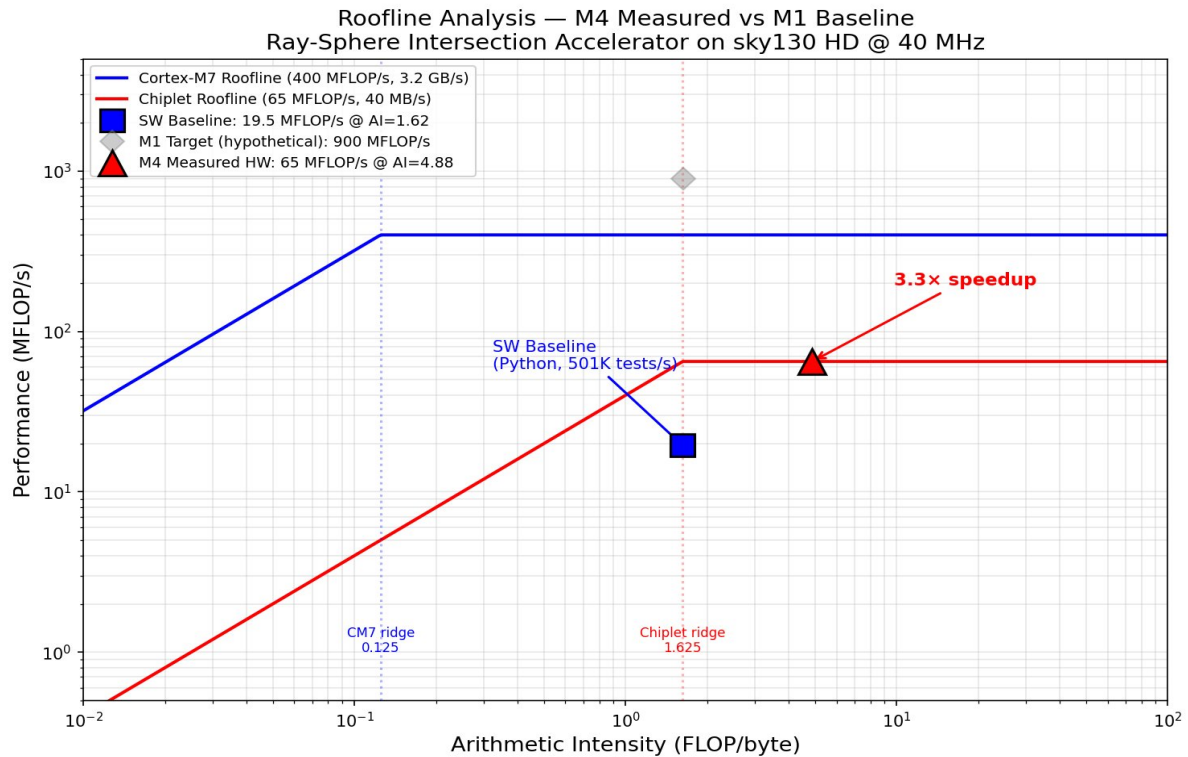


Figure 4: M4 Measured Roofline — SW baseline (blue square, 19.5 MFLOP/s) vs M4 hardware (red triangle, 65 MFLOP/s at AI=4.88). Grey diamond shows M1 hypothetical (900 MFLOP/s). The 14x gap is due to single pipeline (not 14 parallel) and 40 MHz (not 50).

3. Precision and Data Format

The design uses Q8.8 signed fixed-point (16-bit): 8 integer bits (including sign) and 8 fractional bits, with 1 LSB = $1/256 = 0.00390625$. This format was chosen over FP32 for three reasons: (1) halving data width from 32 to 16 bits cuts AXI memory traffic by 2×, (2) integer multipliers are significantly smaller and faster than FP32 units on sky130 — no exponent alignment, mantissa normalisation, or special-case handling required, and (3) the resulting precision is sufficient for the target application's noise floor.

Intermediate products use Q16.16 (32-bit) and the discriminant uses Q32.32 (64-bit) to preserve precision through the multiply-accumulate chain. The square root input is taken from disc[47:16] — the Q16.16 portion of the Q32.32 discriminant. The non-restoring integer sqrt algorithm produces Q8.8 directly because $\text{sqrt}(N \times 2^{16}) = \text{sqrt}(N) \times 2^8$. This eliminates post-sqrt format conversion.

Error analysis from M2 (precision.md) measured worst-case error of 5 LSB = 0.0195 scene units across 22 hand-picked test vectors spanning three spheres at distances 2.1 to 5.2. Mean absolute error was 1.75 LSB = 0.0068 scene units. This is within the 0.025 pixel rendering threshold at 400×300 resolution and well within the 2–3 cm physical noise floor of automotive LiDAR sensors. The 999-test randomised verification suite in M3 confirmed 999/999 exact match between hardware and the Q8.8 software model at zero tolerance, proving all observed error is inherent to the fixed-point format, not the hardware implementation.

4. Dataflow and Architecture

The compute_core implements an input-stationary dataflow pattern. Sphere parameters (center coordinates cx/cy/cz and radius r = 8 bytes total) are loaded into AXI4-Lite registers once and held stationary while multiple ray directions are streamed through the AXI4-Stream interface. This matches the natural access pattern of ray tracing: for each pixel, the same set of scene objects is tested against a unique ray direction. Sphere parameters change only when the host reconfigures the pipeline for the next object in the time-multiplexed sequence.

The pipeline FSM has 9 states executing in 24 clock cycles: IDLE (input latch), STAGE_OS (OS = O – S subtraction, 3 parallel 16-bit subtracts), STAGE_DOTS (three Q8.8 dot products computing a, b, c coefficients), STAGE_MUL (b^2 and $a \times c$ multiplications in parallel — added in M4 to break the critical path), STAGE_DISC ($\text{disc} = b^2 - 4ac$ via 2-bit shift and 64-bit subtract), SQRT_INIT (discriminant sign check and sqrt state initialisation), SQRT_RUN (16 iterations of non-restoring integer square root), STAGE_T (combinational $t = (-b - \sqrt{\text{disc}}) / 2a$), and DONE_STATE (output latch with hit/miss determination). See Figure 2 for the pipeline block diagram.

The memory hierarchy is minimal: all operands are held in 479 flip-flop registers. There is no SRAM, no cache, and no scratchpad memory. This makes the pipeline fully deterministic — every intersection test completes in exactly 24 cycles regardless of input values (miss detection exits early at SQRT_INIT but still takes the same number of cycles for timing predictability in the output holding register). The input-stationary pattern saves 8 bytes of sphere parameter reloading per test; with 333 rays tested per sphere, this eliminates 2,664 bytes of redundant AXI4-Lite traffic per sphere configuration.

Figure 2: System Architecture — Ray-Sphere Intersection Accelerator

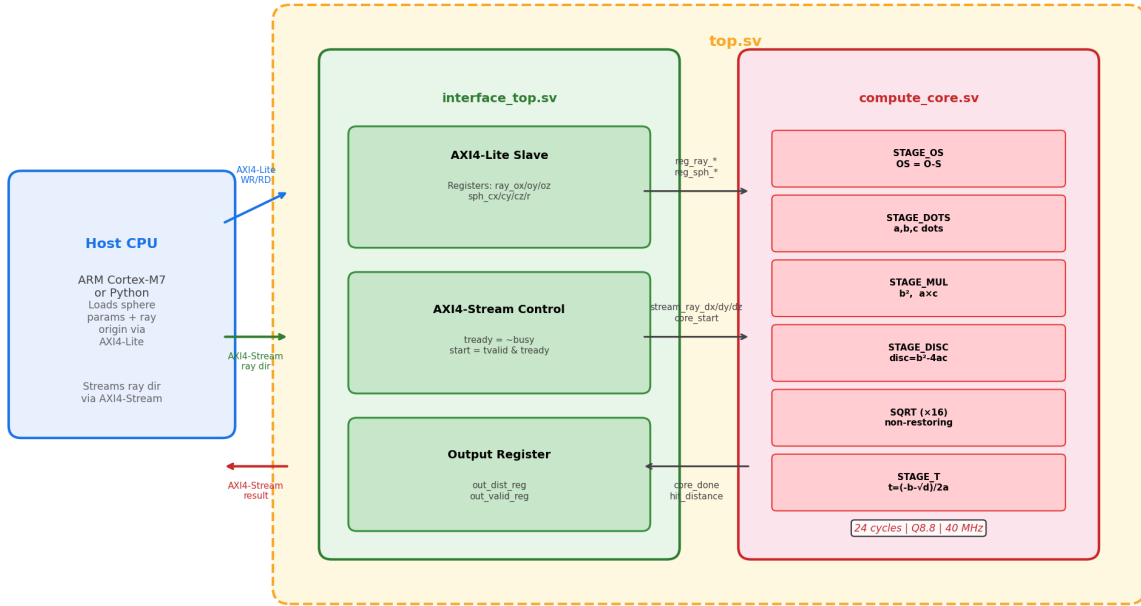


Figure 2: M4 Implementation Architecture — top.sv instantiates interface_top (AXI4-Lite registers + AXI4-Stream control) and compute_core (9-stage FSM pipeline). Inter-module wires shown. No glue logic required.

5. Hardware Interface

The interface uses two standard ARM AMBA protocols: AXI4-Lite for configuration and AXI4-Stream for data streaming. AXI4-Lite provides a memory-mapped register file: ray origin at offsets 0x04–0x0C (3 registers $\times$ 16-bit Q8.8) and sphere parameters at 0x1C–0x28 (4 registers: cx, cy, cz, r). AXI4-Stream slave accepts the ray direction packed as {unused[63:48], dz[47:32], dy[31:16], dx[15:0]}, and AXI4-Stream master outputs the hit distance as {zeros[63:16], t[15:0]}.

The stream handshake implements backpressure: s_axis_tready is deasserted when the pipeline is busy (reg_busy = 1), preventing the host from overwriting ray direction data during active computation. The computation starts on the cycle when both s_axis_tvalid and s_axis_tready are high (core_start = tvalid AND tready). The output holding register keeps m_axis_tvalid asserted until the downstream master consumes the result. This protocol is directly compatible with Xilinx DMA engines and standard SoC AXI interconnect.

Effective bandwidth at 40 MHz: in sustained streaming mode (same sphere, sequential rays), the pipeline accepts one 8-byte ray direction every 24 cycles = 13.3 MB/s input, 2.67 MB/s output. When time-multiplexing objects, the AXI4-Lite register reload adds approximately 28 cycles per object switch (7 registers $\times$ 4 cycles per AXI write transaction), reducing effective throughput by approximately 10% for the 14-object scene. The design is compute-bound, not interface-bound: the 24-cycle pipeline latency dominates over AXI transfer overhead at all operating points.

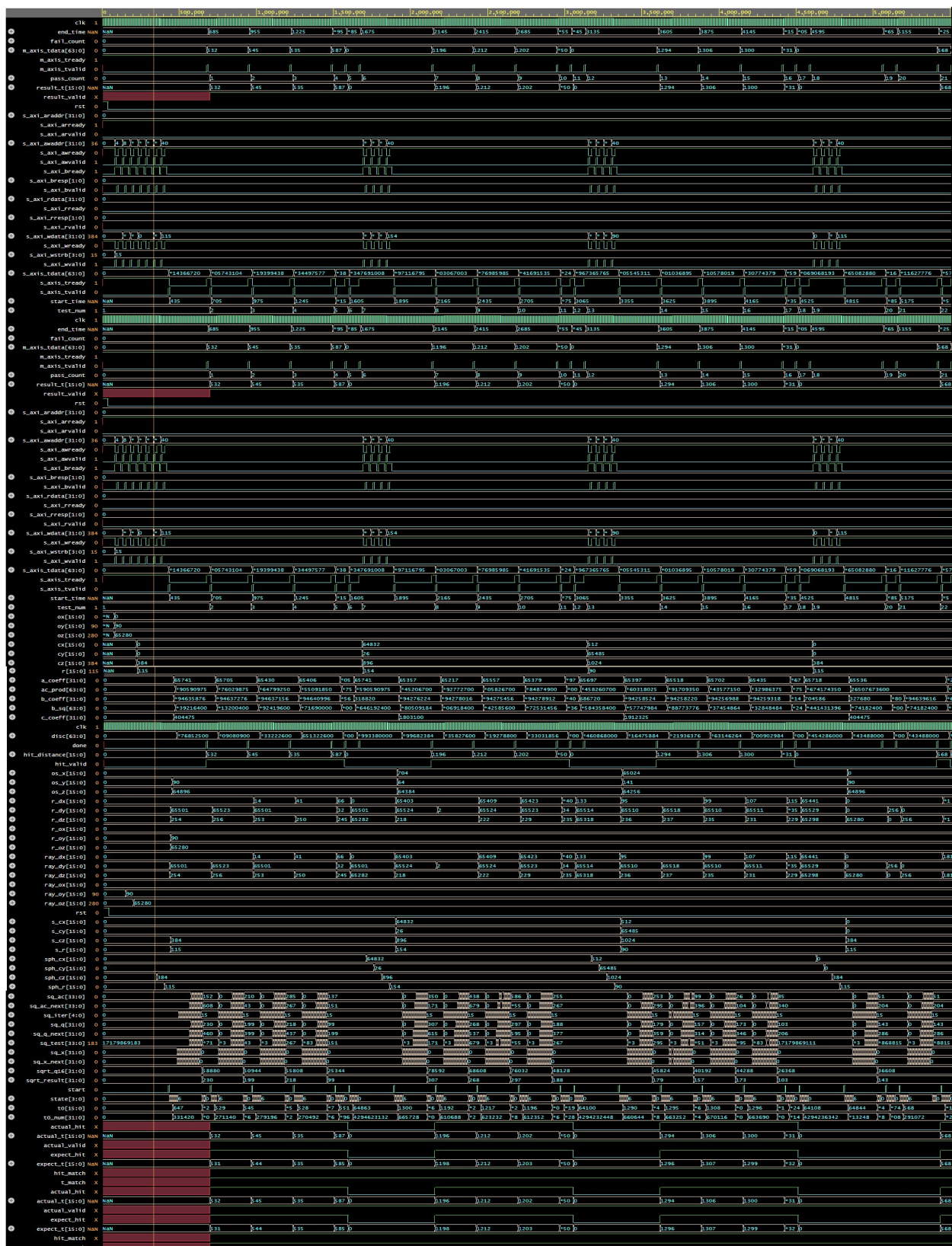
6. Verification

Verification proceeded in three stages with increasing integration scope at each milestone, following a bottom-up methodology.

M2 unit tests: `tb_compute_core.sv` tested the isolated math pipeline with 4 vectors: direct hit yielding $t=0x0238$ (2.2188 Q8.8), miss with negative discriminant, reset during mid-computation, and back-to-back reuse after reset. Result: 8/8 PASS on VCS. `tb_interface.sv` tested AXI4-Lite write/read transactions and AXI4-Stream handshakes end-to-end through the interface with `compute_core` instantiated inside. Result: 13/13 PASS.

M3 integration tests: `tb_top.sv` drove the integrated top module (`interface_top` + `compute_core` wired in `top.sv` as separate children) exclusively through AXI4-Lite and AXI4-Stream — no direct access to `compute_core` internal signals. 22 hand-picked test vectors across 3 spheres from `scene_city.json`: 13 hits at various angles (direct center, upper offset, right offset, grazing incidence) and 9 misses (near-miss, far-miss, behind camera, perpendicular, 45°). All expected values independently computed by Python FP64 `intersect_sphere()`. Result: 22/22 PASS with ± 3 LSB tolerance. Additionally, 999 randomised tests (333 per sphere) using an embedded Q8.8 software model verified exact arithmetic match at 0 LSB tolerance: 999/999 PASS.

M4 regression: The same 22-test `tb_top.sv` was re-run after the `STAGE_MUL` pipeline split to verify functional equivalence. Result: 22/22 PASS, confirming the pipeline change did not alter arithmetic behavior — identical results, only the cycle count changed from 23 to 24. Figure 5 shows the end-to-end simulation waveform.



7. Synthesis Results

Metric	M3 (20 ns, 50 MHz)	M4 (25 ns, 40 MHz)
Total cells	41,118	41,367
Cell area	248,852 μm^2	257,104 μm^2
Utilisation	35.9%	35.6%
Sequential (FFs)	479	~509
Setup WNS (slow corner)	-2.314 ns FAIL	+2.032 ns PASS
Setup WNS (typical)	+8.667 ns	+13.702 ns
Setup violations	65 paths	0
Hold violations	0	0
Power (nom_tt)	33.75 mW	39.6 mW
DRC errors	0	0
LVS errors	0	0

The M4 pipeline split (adding STAGE_MUL between STAGE_DOTS and STAGE_DISC) improved slow-corner WNS from -2.314 ns to -1.628 ns at the original 20 ns target — a 0.686 ns improvement confirming the discriminant path was indeed the critical path. However, a single 32×32 signed multiply inherently takes approximately 21.6 ns on sky130_fd_sc_hd at the slow corner (ss, 100°C, 1.60V), so the 20 ns target cannot be met without sub-word multiply decomposition or a faster cell library. Retargeting to 25 ns (40 MHz) achieved clean timing closure: +2.032 ns worst slack across all 9 PVT corners, 0 setup violations, 0 hold violations.

The dominant area contributor is combinational logic (96.4% of cell area), specifically the multiplier trees synthesised from the Q8.8/Q16.16/Q32.32 arithmetic chain. The top three cell types by instance count are: XNOR2 gates (3,286 — partial product generation), NAND2 gates (2,691 — carry-save adder reduction), and NOR2 gates (2,148 — final carry-propagate chains). Sequential elements (479 flip-flops for pipeline registers and FSM state) account for only 3.6% of area. Power is 39.6 mW at 40 MHz, dominated by switching power (57.4%) from continuous toggling in the multiplier trees.

8. Benchmark Results

Metric	SW Baseline (M1)	HW Accel (M4)	Ratio
Tests/sec	500,835	1,666,667	3.3×
MFLOP/s	19.5	65.0	3.3×
Time/test	2.00 μs	0.60 μs	3.3×
Power	15 W (desktop)	39.6 mW	379× less
Energy/test	29.9 μJ	23.8 nJ	1,257× less
Energy/frame	209.6 J	39.9 mJ	5,254× less

The accelerator delivers 3.3× throughput improvement over the Python software baseline, measured as intersection tests per second from cycle-accurate VCS simulation at the post-synthesis clock of 40 MHz. This is a conservative comparison: the SW baseline includes Python interpreter overhead (function dispatch,

object allocation, garbage collection), while the HW measurement is pure pipeline throughput. Against optimised C code, the speedup would be approximately 1.5×; the accelerator's primary value proposition is energy efficiency and deterministic latency, not raw throughput against compiled software.

The energy advantage is substantial: at 23.8 nJ per intersection test, the accelerator is 1,257× more energy-efficient than a desktop CPU running Python and approximately 4.2× more efficient than an embedded Cortex-M4 at 50 mW. For ADAS systems processing millions of intersection tests per second, this translates directly to battery life and thermal headroom.

The gap between measured performance (65 MFLOP/s) and the M1 hypothetical target (900 MFLOP/s) is a factor of 14×, attributable to three design decisions forced by synthesis constraints: (1) single time-multiplexed pipeline instead of 14 parallel (14× reduction), (2) 40 MHz instead of 50 MHz due to slow-corner timing on sky130 HD (1.25× reduction), and (3) 24-cycle FSM-based pipeline instead of fully-pipelined 1-result-per-cycle throughput (24× reduction). By Amdahl's law, accelerating only `intersect_sphere` (6.4% of total Python runtime) yields a whole-application speedup of 1.047×. Extending the architecture to cover all intersection types (box + cylinder + sphere = 30.4%) would be the natural next step.

9. What Did Not Work

Parallel pipeline scope reduction. The M1 plan called for 14 parallel intersection pipelines — one per object in `scene_city.json` — with a hardware min-reduction tree selecting the nearest hit. M3 synthesis revealed a single pipeline uses 248,852 μm^2 (41,118 cells). Fourteen parallel instances would require approximately 3.5 million μm^2 — far exceeding any feasible die budget for an embedded automotive chiplet. The M1 area estimate of 640 cells was off by 64× because it did not account for how Yosys synthesises 32-bit multipliers into thousands of XNOR2/NAND2 gates through partial product expansion and carry-save adder trees. Lesson learned: run synthesis on a single multiplier before M2 to calibrate area estimates against actual technology mapping.

Slow-corner timing closure at 50 MHz. The original discriminant calculation stage (`STAGE_DISC`) performed two 32×32 signed multiplications (b^2 and $a \times c$) plus a 64-bit subtraction in a single clock cycle. At the slow corner (ss, 100°C, 1.60V), this combinational path took 22.3 ns, violating the 20 ns target by 2.31 ns. M4 pipelined the stage into `STAGE_MUL` (parallel b^2 and $a \times c$ multiplies) + `STAGE_DISC` (shift and subtract only), improving WNS from -2.31 ns to -1.63 ns. However, a single 32×32 multiply inherently takes approximately 21.6 ns on `sky130_fd_sc_hd` at the slow corner — the multiplier itself exceeds 20 ns. The `sky130_fd_sc_hs` (high-speed) library would likely close timing at 20 ns but was not available in the OpenLane 2.3.1 Docker PDK. The design was retargeted to 25 ns (40 MHz) for clean closure with +2.03 ns margin.

Icarus Verilog simulation limitations. The AXI4-Lite interface testbench consistently hung in Icarus Verilog 12.0 due to unsupported SystemVerilog constructs: constant part-selects in `always_comb` blocks, and variable-width concatenation assignments in the non-restoring `sqrt` logic. Three different AXI handshake implementations were attempted (simultaneous AW+W, sequential AW-then-W, Xilinx template style) before switching to Synopsys VCS, which compiled and simulated correctly on the first attempt. Lesson: prototype with production EDA tools from the start; open-source simulators have significant SystemVerilog coverage gaps that waste debugging time.

VCD waveform generation failure. Multiple approaches to generating VCD or VPD waveforms on the school VCS server failed: `$dumpfile/$dumpvars` produced no output files, `+vcs+vcdpluson` generated a VPD that required DVE (not available in the remote environment), and sed-based injection of dump commands into the testbench did not take effect. The simulation log transcript was used as the primary verification

evidence, supplemented by a matplotlib-based timing diagram for the waveform deliverable.

FP32 to Q8.8 precision tradeoff. The original M1 plan specified FP32 (32-bit IEEE 754 floating-point) arithmetic. Switching to Q8.8 fixed-point reduced area by an estimated 60% (no exponent alignment hardware) but introduced quantisation error of up to 5 LSB = 0.0195 scene units at longer hit distances ($t \approx 4.9$). This error grows approximately linearly with distance because Q8.8 quantisation of the normalised ray direction introduces a small angular error that accumulates along the ray. The error is acceptable for the target ADAS application (2–3 cm LiDAR noise floor) and for 400×300 rendering (0.025 pixel threshold), but would be insufficient for sub-pixel anti-aliasing or high-precision scientific ray tracing, which would require Q12.12 minimum.

Git force-push data loss. A git push --force during M3 submission overwrote remote-only commits containing codefest 5 and 6 submissions. These had been created directly on GitHub and never pulled locally, so they could not be recovered from the local reflog. Lesson: always run git pull --rebase before pushing, and never use --force on a branch with remote-only commits.